

Reporte de la práctica

Control por voz de luces y movimiento con Arduino UNO R4 WiFi

1 Datos generales

Institución	Instituto Tecnológico de Mazatlán (TecNM)
Materia	Sistemas Programables
Alumno	Carlos Tadeo Ibarra Castañeda
Actualización	15 de septiembre de 2026
Placa y programas	Arduino UNO R4 WiFi, Arduino IDE y MIT App Inventor en Android
Materiales del circuito	Puente H L298N, motorreductor de corriente directa con rueda, LED externo, resistencia limitadora, protoboard y cables de conexión
Alimentación	Arduino por USB; batería rectangular Energizer Max de 9 V para las pruebas del motor
Comunicación	Wi-Fi de 2.4 GHz; peticiones HTTP GET al servidor del Arduino en el puerto 80

2 Objetivo

Controlar por voz el LED externo, la matriz integrada y el avance, retroceso y paro de un motorreductor en tres niveles PWM. Se confirmó el encendido de las luces y, en la nueva prueba, una IP válida y el registro de baja, media, máxima y paro. Las evidencias permiten evaluar la comunicación y la selección de niveles; falta documentar la respuesta física del motor.

3 Conexiones del circuito

Asignación de conexiones según el programa desarrollado. La revisión física del puente H forma parte del diagnóstico pendiente.

Elemento	Conexión	Función
LED externo	D4, resistencia en serie y ánodo; cátodo a GND	Encendido y apagado
ENA del L298N	D9 del Arduino	PWM; retirar el jumper de ENA
IN1 del L298N	D8 del Arduino	Selección del sentido de giro
IN2 del L298N	D7 del Arduino	Selección del sentido de giro
GND común	Arduino, L298N y negativo de la batería	Referencia eléctrica compartida
OUT1 y OUT2	Terminales del motorreductor	Salida del canal A del puente H
Entrada de potencia	Positivo de la batería al borne de alimentación del motor del L298N	Fuente externa del motor
Matriz integrada	Sin cableado externo	Control con Arduino_LED_Matrix

La alimentación lógica del módulo y su jumper 5V-EN deben revisarse según la versión del L298N. No se registraron el voltaje nominal ni la corriente de arranque del motor.

4 Procedimiento

- 1 Se tomó como base el proyecto de App Inventor compartido y se revisaron los componentes Voz, Habla, Web1, BtnHablar y las etiquetas de comando y estado.
- 2 Se configuraron D4 para el LED externo y D9, D8 y D7 para el puente H. El programa inicia el motor detenido y la matriz apagada mediante `matriz.begin()` y `matriz.clear()`.
- 3 Se programó la conexión Wi-Fi con WiFiS3 y se consultó el Monitor Serie a 9600 baudios. En la nueva prueba se obtuvo `http://192.168.100.42`, dirección que también aparece en el bloque de IP de la app.
- 4 Al pulsar BtnHablar, `Voz.GetText` obtiene el reconocimiento de voz. El evento `AfterGettingText` compara el texto con las frases previstas y llama al procedimiento `enviar`.
- 5 El procedimiento une la IP y la ruta, asigna el resultado a `Web1.Url` y ejecuta `Web1.Get`. El Arduino interpreta la petición y responde con un mensaje de texto. [1]
- 6 Se añadieron las rutas de avance, retroceso y paro para coincidir con la app. Se registró la secuencia baja, media, máxima y detener; sus datos se organizaron en una tabla y dos gráficas.

Comandos y rutas utilizados

Comando de voz	Ruta HTTP	Acción programada
Prende el LED	/led/on	D4 en HIGH
Apaga el LED	/led/off	D4 en LOW
Prende la matriz	/matriz/on	Mostrar 8 filas de 12 LED
Apaga la matriz	/matriz/off	Limpiar la matriz
Avanza velocidad baja	/motor/avanza/baja	Avance con PWM 110
Avanza velocidad media	/motor/avanza/media	Avance con PWM 180
Avanza velocidad máxima	/motor/avanza/maxima	Avance con PWM 255
Retrocede velocidad baja	/motor/retrocede/baja	Retroceso con PWM 110
Retrocede velocidad media	/motor/retrocede/media	Retroceso con PWM 180
Retrocede velocidad máxima	/motor/retrocede/maxima	Retroceso con PWM 255
Detener motor	/motor/detener	ENA sin activación

Las rutas se escriben sin acentos. Se recomienda convertir el texto reconocido a minúsculas y normalizar acentos antes de compararlo. La versión inicial del Arduino sólo atendía `/vel/0` a `/vel/3`, por lo que no interpretaba las rutas `/motor/...` que enviaba la app.

5 Resultados

La nueva evidencia del Monitor Serie registra la conexión con IP válida y las órdenes de avance en los tres niveles, seguidas del paro. Los datos siguientes se transcribieron de esa captura; describen el estado que reporta el programa.

Tabla de datos de la prueba

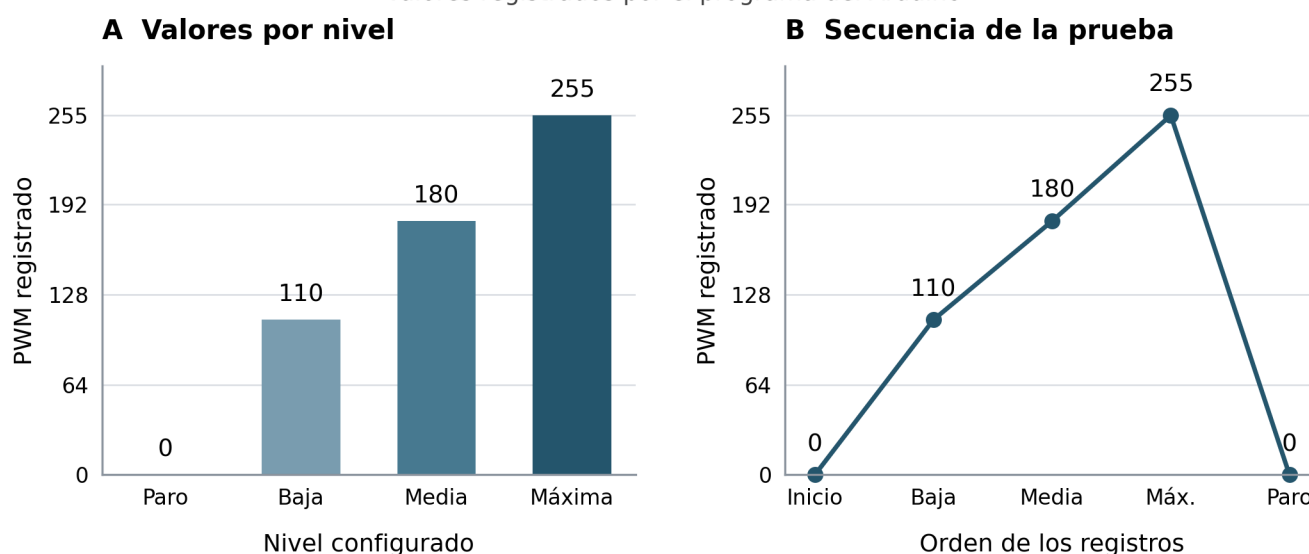
Paso	Orden o estado	Nivel	PWM	Ciclo aprox.	Respuesta registrada
0	Inicio	0	0	0 %	Dirección 0 y nivel 0
1	Avance bajo	1	110	43.1 %	Avanzando velocidad baja
2	Avance medio	2	180	70.6 %	Avanzando velocidad media
3	Avance máximo	3	255	100 %	Avanzando velocidad maxima
4	Detener	0	0	0 %	Motor detenido

En la escala de 8 bits, el ciclo de trabajo aproximado se obtiene al dividir PWM entre 255 y multiplicar por 100. [2] Los porcentajes son calculados; no se dispone de mediciones con osciloscopio.

Gráficas de los resultados

Resultados de control PWM

Valores registrados por el programa del Arduino



Fuente: Monitor_Serial.png. La gráfica B no tiene escala temporal.

Los valores PWM no son mediciones de velocidad física del motor.

Figura 1. Comparación de niveles y secuencia de los valores PWM registrados.

La gráfica A permite comparar los valores de cada nivel. La gráfica B conserva el orden de los mensajes: estado inicial, baja, media, máxima y paro. La separación horizontal representa eventos consecutivos, porque la captura no incluye marcas de tiempo.

Se observa un incremento de 70 unidades PWM de baja a media y de 75 de media a máxima. El comando detener devuelve el valor a 0. Esto confirma la selección de los niveles en el software; no permite establecer las revoluciones por minuto ni la diferencia real de velocidad bajo carga.

6 Salida del Monitor Serie

El registro corresponde a la versión identificada como CONTROL VOZ - IP V3. Se muestran la conexión y cuatro peticiones HTTP del motor, cada una con su nivel y respuesta.

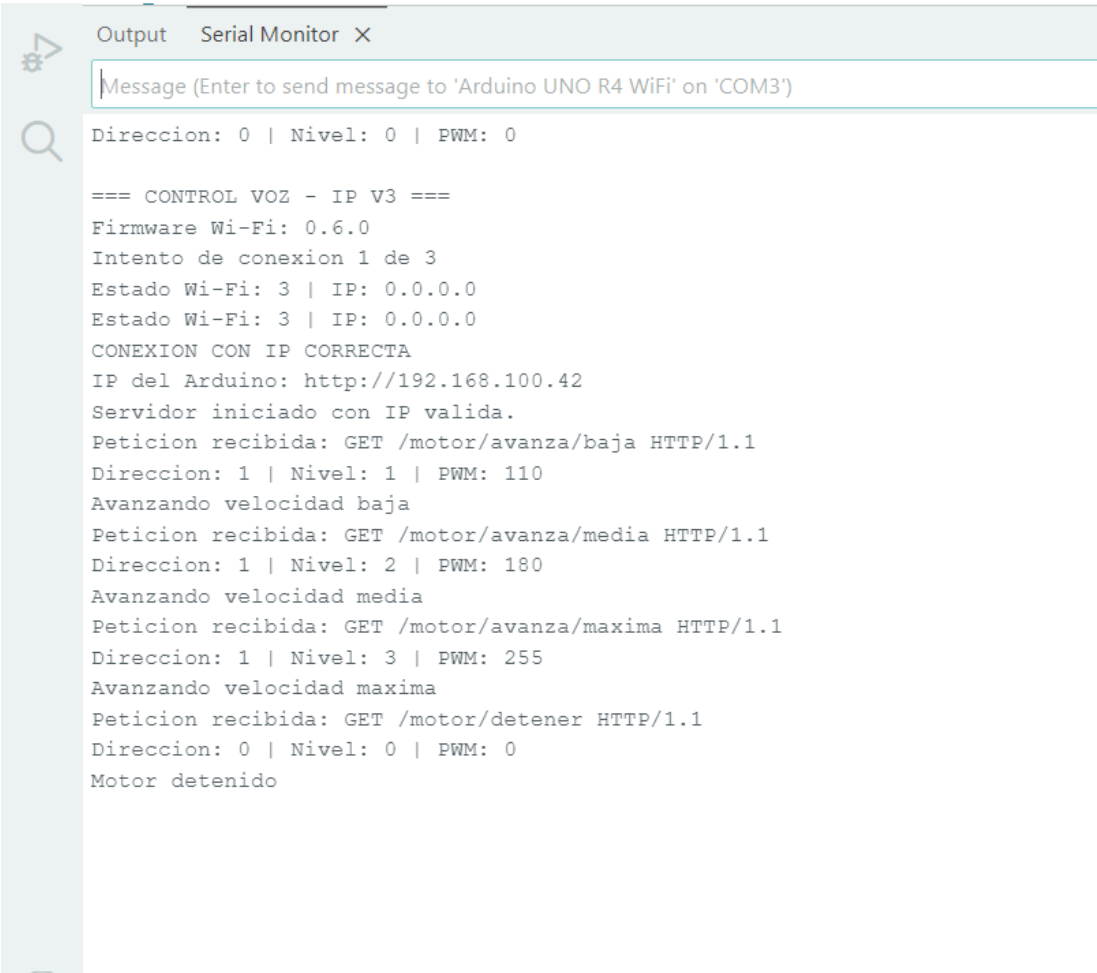


Figura 2. Evidencia actualizada de conexión y recepción de las órdenes del motor.

Dato del registro	Valor observado
Firmware Wi-Fi	0.6.0
Intento de conexión	1 de 3
IP asignada	192.168.100.42
Secuencia de PWM	110, 180, 255 y 0 después del estado inicial

Al principio se imprime 0.0.0.0 en dos consultas. Después aparece CONEXION CON IP CORRECTA y el servidor se inicia con 192.168.100.42. En esta prueba se resolvió el problema de la dirección no utilizable.

Las rutas de retroceso están programadas en la app, pero no aparecen en esta secuencia del Monitor Serie. Su recepción y efecto físico deben comprobarse en una prueba adicional.

7 Observaciones sobre el comportamiento del sistema

Correspondencia de rutas. En los primeros bloques se enviaba `/led/of` y `/matriz/of`, mientras el Arduino esperaba `/led/off` y `/matriz/off`. También existió una diferencia entre las rutas `/motor/...` de la aplicación y `/vel/...` del sketch. Se ajustó el código para que ambos extremos reconocieran las mismas órdenes.

Conexión Wi-Fi. En los primeros intentos sólo aparecían puntos o la dirección 0.0.0.0. La nueva captura muestra que V3 espera la IP y después inicia el servidor. La coincidencia de 192.168.100.42 entre el Monitor Serie y la app permite dirigir las peticiones a la placa en esa red.

Respuesta a los niveles. La secuencia baja, media y máxima produjo los valores 110, 180 y 255 en el registro. Al solicitar detener, dirección y nivel regresaron a 0. Se verificó esta respuesta del programa, aunque no se registraron rpm ni voltajes en los terminales del motor.

Motor y batería. En las pruebas iniciales el motor no giró aun con PWM 255 y se utilizó una Energizer Max rectangular de 9 V. Se consideró una posible caída de tensión al arrancar, sin mediciones que confirmaran esa causa. La nueva captura acredita los comandos, pero no muestra el movimiento físico. Deben comprobarse la alimentación, tierra común, ENA y las salidas del puente H. El L298N también presenta una caída de tensión interna. [3]

Confirmación en la aplicación. En los bloques mostrados, la app dice la respuesta antes de recibir la contestación HTTP. Por ello, escuchar “Avanzando” sólo confirma la selección del comando. Como mejora, debe actualizarse el estado desde Web1.GotText y verificarse `responseCode`; aun así, la respuesta del Arduino no equivale a una medición del movimiento. [1]

Reconocimiento y retroceso. Los bloques contienen avance y retroceso en tres niveles, además de paro. El texto se compara con frases específicas, por lo que conviene normalizar mayúsculas y acentos. La captura serie únicamente documenta avance y paro; aún falta una evidencia equivalente para retroceso.

Estado de las funciones al cierre de las pruebas

Función	Estado registrado
LED externo	Encendido confirmado; apagado programado y petición <code>/led/off</code> registrada.
Matriz LED	Encendido confirmado; petición <code>/matriz/off</code> registrada. Falta confirmación física final del apagado.
Motorreductor	Registro confirmado de baja, media, máxima y paro. Falta documentar movimiento físico y retroceso.
Conexión final	IP 192.168.100.42 y servidor iniciado correctamente en la nueva prueba.

Para completar la validación física se requiere medir la alimentación durante el arranque, comprobar ambos sentidos y registrar el comportamiento en cada nivel. La prueba inicial debe realizarse con la rueda levantada.

8 Evidencias de la aplicación y el circuito

Diseño de la aplicación

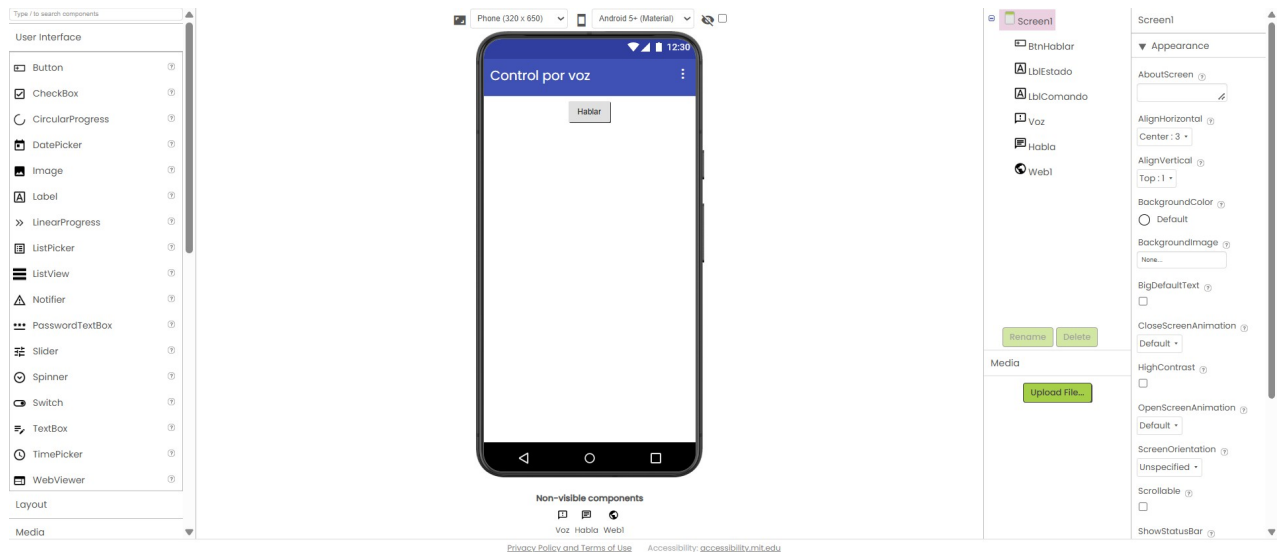


Figura 3. Pantalla de la aplicación y componentes de App Inventor.

La interfaz concentra el uso en el botón Hablar. LblComando muestra la orden seleccionada y LblEstado el mensaje correspondiente. Voz, Habla y Web1 realizan el reconocimiento, la respuesta hablada y la comunicación HTTP.

Representación del circuito en Tinkercad

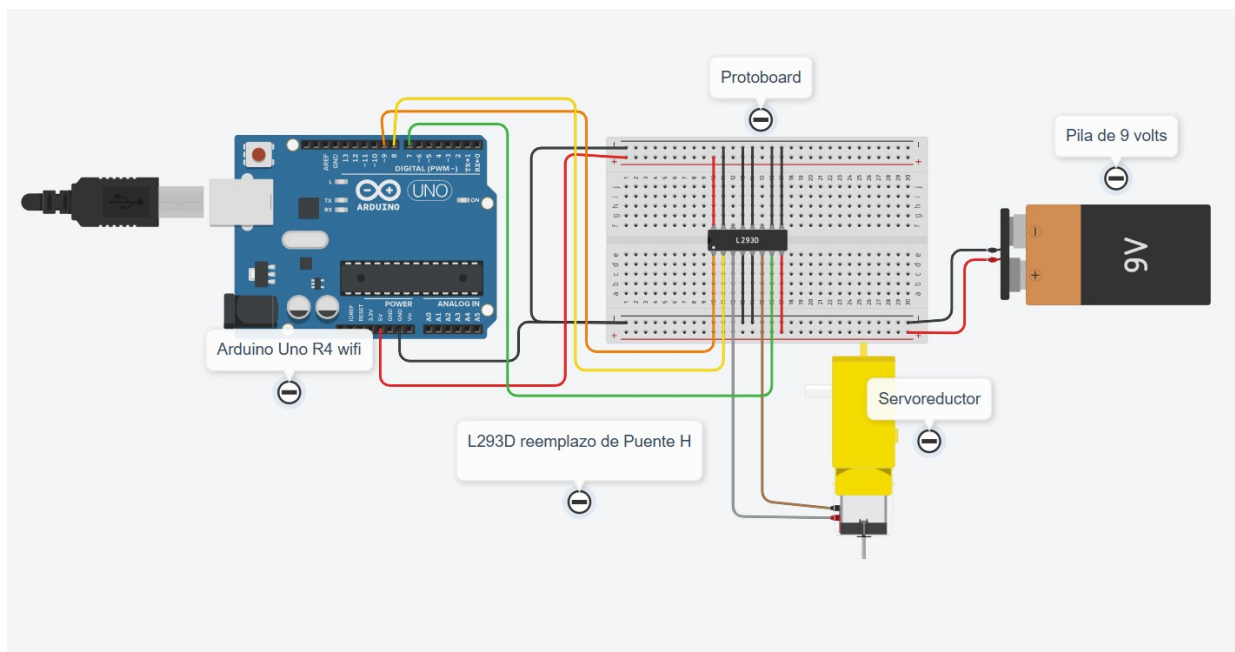


Figura 4. Representación de la etapa del motor en Tinkercad con L293D.

La imagen usa un Arduino UNO convencional y un L293D para representar la etapa del motor; la práctica física utiliza UNO R4 WiFi y L298N. El componente rotulado “Servoreductor” es un motorreductor de corriente directa. El esquema no documenta el control por voz ni incorpora el LED externo o la matriz. Cada controlador requiere su propio cableado.

Bloques para la comunicación y las luces

Las capturas actualizadas muestran la IP de la prueba, el procedimiento enviar y las condiciones que reconocen órdenes para el LED externo y la matriz.

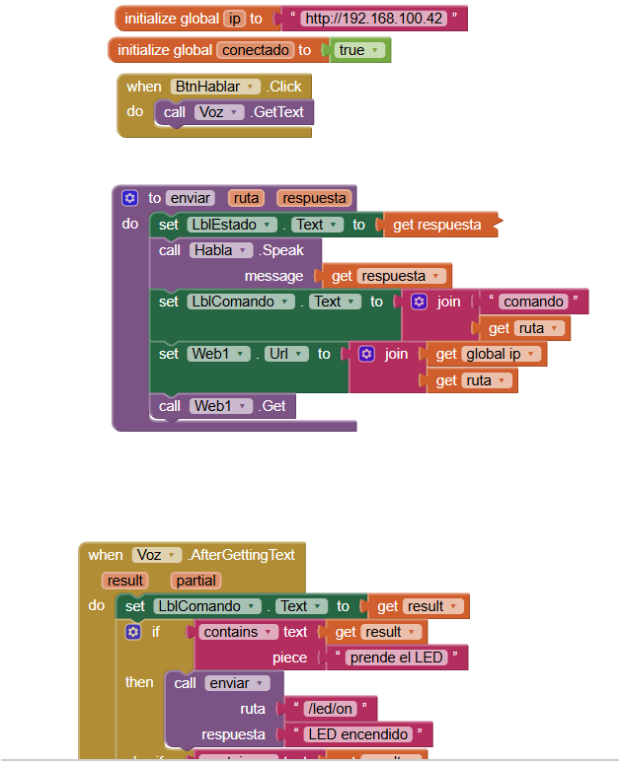


Figura 5. Configuración de IP y envío de peticiones.

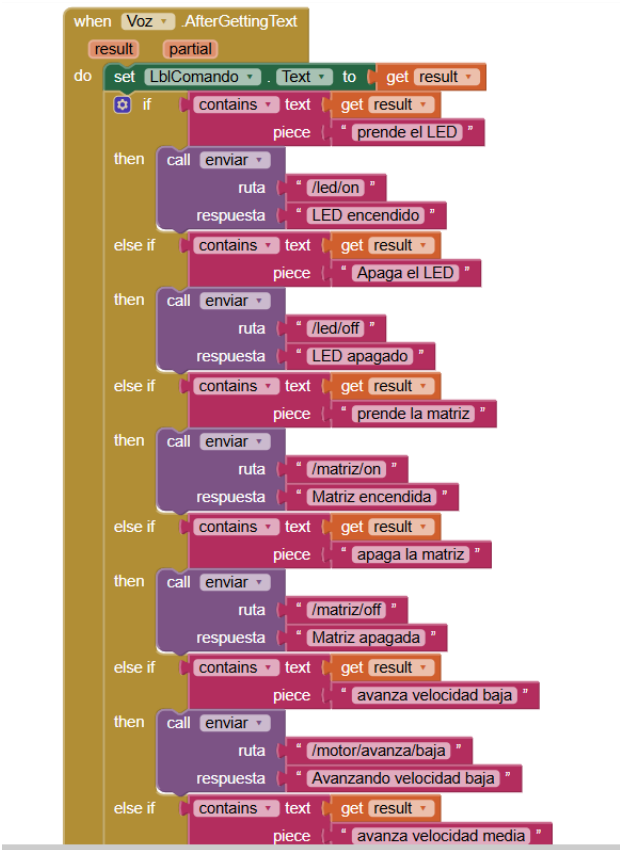


Figura 6. Reconocimiento de órdenes para las luces.

Funciones de los componentes

Componente	Función dentro de la aplicación
BtnHablar	Solicita el reconocimiento de voz mediante Voz.GetText.
Voz	Entrega el texto reconocido en AfterGettingText.
Web1	Envía una petición GET a la IP y la ruta seleccionadas.
Habla	Pronuncia el mensaje de respuesta elegido por la app.
LblComando y LblEstado	Muestran la orden seleccionada y el mensaje de estado.

La variable conectado aparece en true. En el procedimiento mostrado, Web1.Get se ejecuta directamente. El mensaje hablado está antes de la petición; por ello se recomienda confirmar la respuesta HTTP con el evento GotText. [1]

Bloques para el control del motor

Las condiciones asignan una ruta distinta a cada sentido y nivel. La ruta /motor/detener permite solicitar el paro desde el mismo procedimiento enviar.

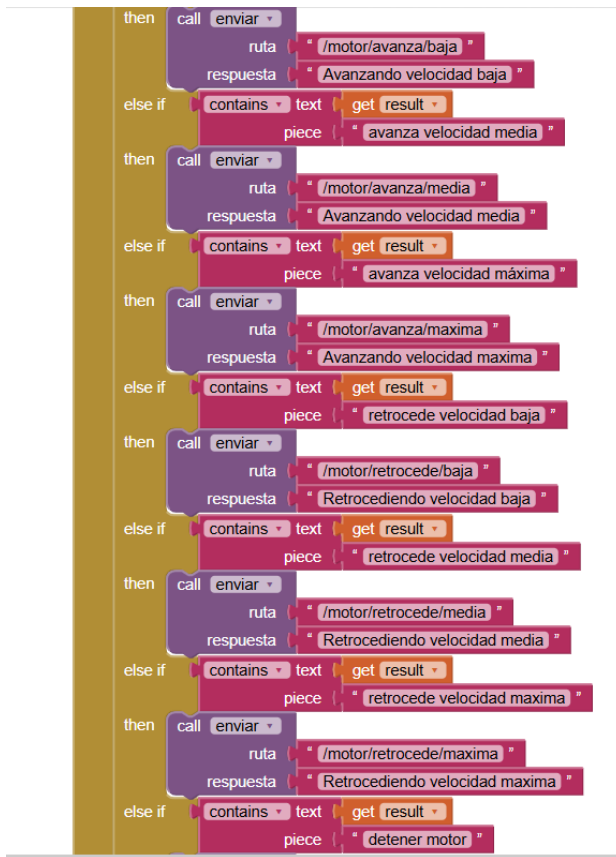


Figura 7. Rutas de avance y retroceso.

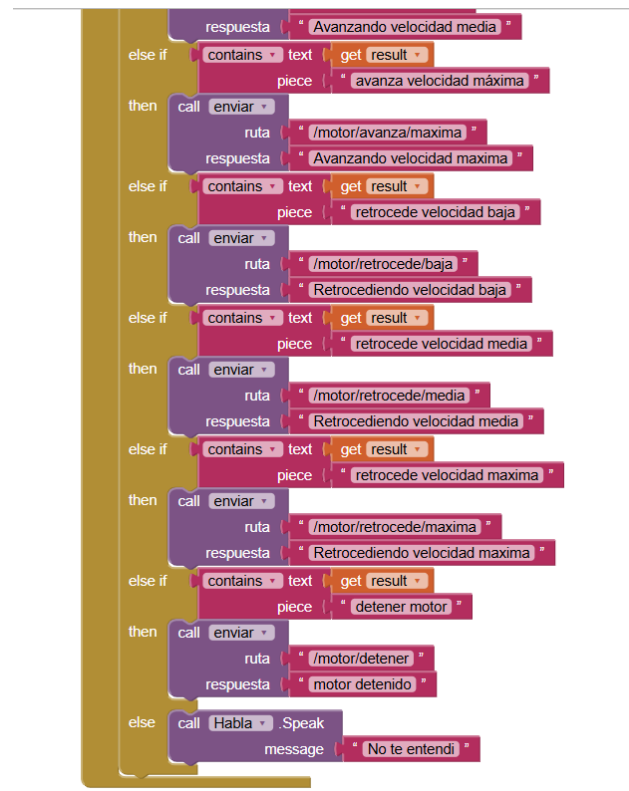


Figura 8. Rutas de retroceso y orden de paro.

9 Conclusión

La práctica permitió controlar las luces por voz y comprobar la comunicación HTTP con el Arduino. La nueva prueba resolvió la obtención de IP y registró los niveles PWM 110, 180 y 255, seguidos de 0 al detener. Las tablas y gráficas muestran que el programa seleccionó los valores correspondientes a las órdenes recibidas.

La validación de comunicación y niveles quedó documentada. Para completar la evaluación del motor falta registrar su giro en ambos sentidos y comparar su velocidad física en cada nivel. El registro serie permite revisar el programa, mientras que las mediciones y la observación de la rueda permiten evaluar la etapa de potencia.

Fuentes técnicas

- [1] MIT App Inventor. Connectivity, componente Web. <https://ai2.appinventor.mit.edu/reference/components/connectivity.html>
- [2] Arduino. analogWriteResolution. <https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWriteResolution>
- [3] STMicroelectronics. L298 Dual full bridge driver, DS0218, revisión 5, 2023. <https://www.st.com/resource/en/datasheet/l298.pdf>